

# Automated Comparison of Protocol Security using Symbolic Verification

## Artefact Evaluation Instructions

Benjamin Bisping <sup>1</sup>, Nicolas Matei<sup>1</sup>, Natalia Kushik <sup>1</sup>, Philipp Schlehuber-Caissier <sup>1</sup>, Chrystel Gaber <sup>2</sup>, and Jean-Philippe Wary <sup>2</sup>

<sup>1</sup> SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris

<sup>2</sup> Orange Innovation, Paris, France

## A Badge Claims

We claim two EAPLS badges:

**Available** The artefact is permanently hosted on Zenodo at <https://zenodo.org/records/21480719>, additionally, the current state of the tool can be found at <https://github.com/benkeks/compareverif>.

**Functional and Reusable** Our artefact exceeds the requirements for the functional badge (see functional outcomes F1-F5 below) and includes reusability claims R1–R3 demonstrating application beyond the scenarios studied in the paper.

Both, a Docker image (as a `.tar.gz` archive) and the full source archive are provided on Zenodo. The Docker image enables reviewers to execute the artefact without installing additional software; the github repository allows reviewers to run the tool directly from source.

## Functional Outcomes

The following functional outcomes correspond to the results presented in the paper.

- F1** Section 2 of the paper demonstrates that hashed passwords are verified as secure against a baseline attacker. (*Outcome*) Running the ProVerif verification on `hashed_passwords_paper.pv` produces two `is true` messages for the two queries of password and hashed-password security. The output confirms the baseline security claim from Section 2.
- F2** Section 4.1 introduces batch processing of attack scenarios. (*Outcome*) The scenario preprocessor enumerates capability combinations and reports that breaking no-pw-leakage requires {Server compromised} (cost: 4 hack) or {Database leak, Rainbow table attack} (cost: 1 hack, 2 time), and breaking no-hashed-pw-leakage requires {Database leak} (cost: 1 hack). These results match the batch processing output described in Section 4.1.
- F3** Section 4.1, end: Pareto front visualization. (*Outcome*) The Pareto comparison tool generates a plot resembling Fig. 3 of the paper, showing cost fronts for breaking queries across attack scenario combinations. The rendered plot confirms the Pareto front shape shown in Fig. 3.

- F4** Section 4.2: Attack tree extraction. (*Outcome*) The attack tree extractor produces a PDF visualization of the attack tree for the `rainbow_table_attack+database_leak` scenario, showing derivation paths from capabilities to the broken query. The attack tree matches the structure described in Section 4.2.
- F5** Section 5.3: Benchmarking singularization (Figs. 5 and 6). (*Outcome*) Running the scenario preprocessor on `singularized_passwords_paper.pv` and the Pareto comparison tool with `--query "no pw leakage"` reproduces the comparison plot from Fig. 5. The attack tree extractor reproduces the attack tree for the singularized model from Fig. 6. Results correspond to the benchmarking figures in Section 5.3.

## Reusability Claims

The following reusability claims demonstrate how the artefact can be used beyond the scenarios covered in the paper.

- R1** *Processing new ProVerif models.* CompareVerif accepts any ProVerif file annotated with the *magical-comment* capability syntax. A new protocol model can be analyzed without modifying the tool, simply by annotating it with `(** Attack Name [price] code **)` blocks. (*Claim*) The following steps demonstrate processing an arbitrary annotated model. Inside the Docker container, the reviewer can inspect the provided example files to see the annotation syntax, then run the preprocessor on any such file. (*Replicable scenario*)

```
docker run -it compareverif bash
# Inspect the annotation syntax:
head -30 examples/hashed_passwords_paper.pv
# Run on the annotated file:
python3 scenario_preprocessor.py examples/hashed_passwords_paper.pv
# Inspect the generated manifest:
cat _scenarios/hashed_passwords_paper/manifest.json
exit
```

The manifest verification results for all capability combinations demonstrate that the tool works on arbitrary models without modification.

- R2** *Analyzing a different protocol: simple ratchet.* (*Claim*) The artefact includes a simple ratchet message-exchange protocol (`examples/simple_ratchet.pv`) and a Jupyter notebook (`notebooks/exhaustive_generation_of_AG_simple_ratchet.ipynb`) that examines attack scenarios beyond those in the paper. (*Replicable scenario*) Reviewers can reproduce its Pareto fronts with a single command:

```
docker run --rm compareverif \
python3 scenario_preprocessor.py examples/simple_ratchet.pv
```

## B Quick Start

### Artefact Hosting

The artefact is available for download from Zenodo:

<https://zenodo.org/records/21480719>

The source code is additionally mirrored at:

<https://github.com/bisping-isp/compareverif>

## Prerequisites

The only prerequisites are **Docker** and **Bash**. (Tested on Debian GNU/Linux 14 (forky), Docker version 28.5.2. and Ubuntu 24.04, Docker version 29.1.3)

## Setting Up the Docker Image

**Option A: Load the provided image.** Download `compareverif-docker.tar.gz` from Zenodo and load it:

```
docker load < compareverif-docker.tar.gz
```

**Option B: Build from source.** Clone the source repository and build:

```
git clone https://github.com/bisping-isp/compareverif.git
cd compareverif
docker build . -t compareverif
```

After either option, verify the image is present:

```
docker image ls
```

The output should list `compareverif` among the available images.

## Sanity Check

Run a basic command to confirm the artefact is operational:

```
docker run --rm compareverif python3 -m pytest
```

This executes the unit test suite and should report all tests passing.

## Directory Structure

The following describes the layout of the submitted artefact (inside the Docker image and in the source archive). Files used in Sections **C** and **D** are marked with (\*).

```
compareverif/
  scenario_preprocessor.py ... Batch processing of attack scenarios
  pareto_comparison.py ..... Pareto front visualization
  attack_tree_extractor.py ... Attack tree extraction from ProVerif output
  compareverif/ ..... Shared library modules
  examples/ ..... ProVerif models
    hashed_passwords_paper.pv      ((* Sec.~2, 4.1)
    singularized_passwords_paper.pv ((* Sec.~5.3)
    singularized_passwords_no_elgamal.pv ($\star$ R3)
    simple_ratchet.pv              ((* R2)
    primitives.pvl                 ((* shared library)
  _scenarios/ ..... Generated scenarios and manifests (after run)
  notebooks/ ..... Jupyter notebooks (R2)
```

```

tests/ ..... Unit tests
  unit/ ..... Unit test modules
  integration/ ..... Integration tests
docs/ ..... Documentation
  manifests.md ..... Manifest format documentation
  attack-trees.md ..... Attack tree format documentation
artefact/ ..... Artefact submission files
  README.md ..... Walk-through instructions
  reproduce-results.sh ..... Push-button reproduction script
Dockerfile ..... Docker image definition
Requirements.txt ..... Python dependencies (matplotlib, pytest)

```

## C Functional Evaluation

All results from the paper can be reproduced by calling

```
./reproduce-results.sh .
```

Each step is detailed within this script and can be executed individually.

## D Reusability Evaluation

This section validates the reusability claims R1 and R2 from Section [A](#).

### D.1 R1: Processing New ProVerif Models

CompareVerif accepts any ProVerif file that uses the *magical-comment* syntax to declare optional attacker capabilities. The syntax is:

```

(** Capability Name [cost] **
 (* ProVerif code to activate this capability *)
***)

```

To verify that the tool generalizes, reviewers can create a minimal ProVerif model with one or two capability blocks and run:

```

docker run -it compareverif bash
# Inside the container, create or edit a .pv file:
# vi examples/my_protocol.pv
python3 scenario_preprocessor.py examples/my_protocol.pv
ls _scenarios/my_protocol/
cat _scenarios/my_protocol/manifest.json
exit

```

The manifest verification results for all capability combinations demonstrate that the tool works on arbitrary models without modification.

### D.2 R2: Analyzing a Different Protocol—Simple Ratchet

The artefact includes a simple ratchet protocol that models message exchange with forward secrecy. This protocol is unrelated to the password-authentication case study in the paper.

```
docker run --rm compareverif \
  python3 scenario_preprocessor.py examples/simple_ratchet.pv
```

**Expected output:** The tool enumerates attack scenarios for the ratchet protocol and reports which security properties hold or break under various capability combinations.

A companion Jupyter notebook (`notebooks/exhaustive_generation_of_AG_simple_ratchet.ipynb`) provides a detailed interactive exploration of attack scenarios, including Pareto fronts for message secrecy at different stages of the ratchet protocol.

This demonstrates how CompareVerif can be used to quantify the security level of design decisions in protocol engineering—a use case that goes beyond the specific protocols studied in the paper.